



ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA DE TELECOMUNICACIÓN

Fundamentos de Inteligencia Artificial

Satisfacción Restricciones

José A. Montenegro

5 de febrero de 2026



Contenido

Introducción

Definición del Problema y Ejemplos

Restricciones y Consistencia

Propagación de Restricciones

- Algoritmo de revisión de dominios

- Algoritmo AC-3

- Ejemplo Sudoku

Algoritmos

- Vuelta Atrás

- Búsqueda Local

Resumen



Introducción





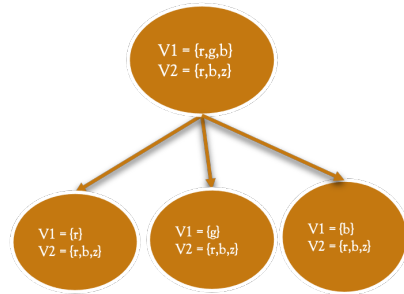
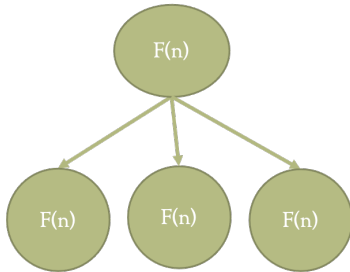
Representaciones factorizadas

- En los temas 4,5 y 6 vamos a explorar problemas que pueden resolverse buscando en un espacio de estados
- Cada estado será atómico, es decir, era una caja negra sin estructura interna.
- Aquí emplearemos una representación factorizada para cada estado
 - Un conjunto de variables, cada una con su valor
 - Un problema está resuelto cuando cada variable tiene un valor que satisface todas las restricciones sobre dicha variable



Representaciones factorizadas

- En los temas 4,5 y 6 evaluaremos los nodos por una función asociada a cada estado ya que son estados atómicos, $F(n)$.
- Ahora los estados contienen asignaciones a las variables.





Definición (I)

- Un problema de satisfacción de restricciones (constraint satisfaction problem, CSP) está formado por tres componentes:
 - Un conjunto de **variables**, $X = \{X_1, \dots, X_n\}$
 - Un conjunto de **dominios**, uno para cada variable: $D = \{D_1, \dots, D_n\}$
 - Un conjunto de **restricciones** que especifican combinaciones permitidas de valores, C
- Cada dominio D_i es el conjunto de valores posibles $\{v_1, \dots, v_k\}$ para la variable v_i .



Definición (II)

- Cada restricción C_i es un par $\langle \text{scope}, \text{rel} \rangle$ donde scope es la tupla de variables que intervienen en la restricción y rel es una relación que define los valores que dichas variables pueden tomar.
 - Por ejemplo, si las variables X_3 y X_5 deben tener valores distintos, podemos escribir esta restricción como $\langle (X_3, X_5), X_3 \neq X_5 \rangle$



Definición (III)

- Cada **estado** de un CSP se define como una asignación de valores a algunas o a todas las variables, $X_i = v_i, X_j = v_j, \dots$
- Una asignación que no viola ninguna restricción se llama **asignación consistente** o legal
- Si tenemos una asignación en la cual todas las variables están asignadas, la llamamos **asignación completa**.
 - En otro caso, la llamamos **asignación parcial**.
- Una **solución** de un CSP es una asignación consistente y completa

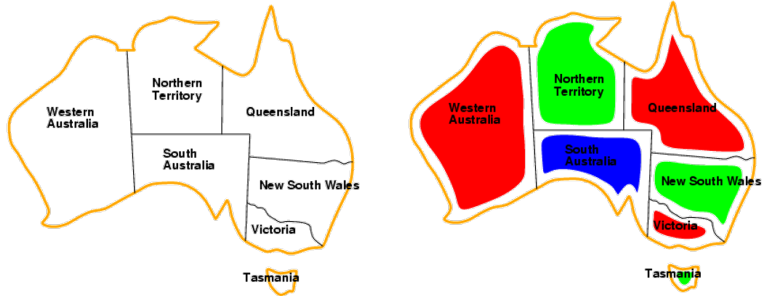


Ejemplo 1: Coloreado de mapas

- La tarea consiste en colorear cada región de un mapa de tal manera que no haya regiones adyacentes que tengan el mismo color
- Para el mapa de Australia (siguiente transparencia), definimos las variables como $X=\{WA, NT, Q, NSW, V, SA, T\}$
- El dominio de cada variable es $D_i = \{red, green, blue\}$
- Hay nueve restricciones: $C=\{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, SA \neq T, WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V\}$



Ejemplo 1: Coloreado de mapas



- Ejemplo de solución completa y consistente:
WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green



Ejemplo 1: Coloreado de mapas





Ejemplo 2: problemas criptoaritméticos

En un acertijo criptoaritmético, cada letra representa a un dígito distinto (0-9)

$$\begin{array}{r} T \quad W \quad O \\ + \quad T \quad W \quad O \\ \hline F \quad O \quad U \quad R \end{array}$$



Ejemplo 2: problemas criptoaritméticos

- El requisito de que todas las variables han de tomar diferentes valores se corresponde con la restricción $AllDiff(F, T, U, W, R, O)$
- Introducimos tres variables auxiliares C_{10} , C_{100} y C_{1000} , que representan los dígitos acarreados a las columnas de las decenas, las centenas y los millares, respectivamente.
- De esta manera el resto de las restricciones son:
 - $O + O = R + 10 * C_{10}$
 - $C_{10} + W + W = U + 10 * C_{100}$
 - $C_{100} + T + T = O + 10 * C_{1000}$
 - $C_{1000} = F$



Ejemplo 2: problemas criptoaritméticos

Solución del acertijo criptoaritmético

- Variables: $T, W, O, U, F, R, C_1, C_2, C_3$

- Dominios:

$$T = W = O = U = F = R = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

$$C_1, C_2, C_3 = \{0, 1\}$$

- Restricciones:

$$\text{AllDiff}(F, T, U, W, R, O)$$

$$R + C_1 * 10 = O + O$$

$$U + C_2 * 10 = W + W + C_1$$

$$O + C_3 * 10 = T + T + C_2$$

$$F = C_3$$

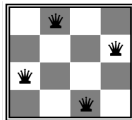
$$\begin{array}{r} T \quad W \quad O \\ + \quad T \quad W \quad O \\ \hline F \quad O \quad U \quad R \end{array}$$



Ejemplo 3: N-Reinas

Una variable por cada casilla

- Variables: X_{ij}
- Dominios: $\{0, 1\}$
- Restricciones:
 - $\forall i, j, k (X_{ij}, X_{ik}) \in \{(0,0), (0,1), (1,0)\} \rightarrow X_{ij} + X_{ik} \leq 1$ (h)
 - $\forall i, j, k (X_{ij}, X_{kj}) \in \{(0,0), (0,1), (1,0)\} \rightarrow X_{ij} + X_{kj} \leq 1$ (v)
 - $\forall i, j, k (X_{ij}, X_{(i+k)(j+k)}) \in \{(0,0), (0,1), (1,0)\} \rightarrow X_{ij} + X_{(i+k)(j+k)} \leq 1$ (d1)
 - $\forall i, j, k (X_{ij}, X_{(i+k)(j-k)}) \in \{(0,0), (0,1), (1,0)\} \rightarrow X_{ij} + X_{(i+k)(j-k)} \leq 1$ (d2)
 - $\sum_{i,j} X_{ij} = N$

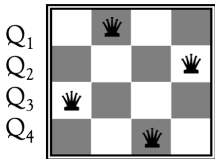




Ejemplo 3: N-Reinas

Una variable por cada fila

- Variables: Q_i
- Dominios: $\{1, 2, 3, \dots, N\}$
- Restricciones:
 - Implícito $\forall i, j$ no ataca (Q_i, Q_j)
 - Explícito Por ejemplo: $(Q_1, Q_2) \in \{(1, 3), (1, 4), \dots\}$





Restricciones y Consistencia





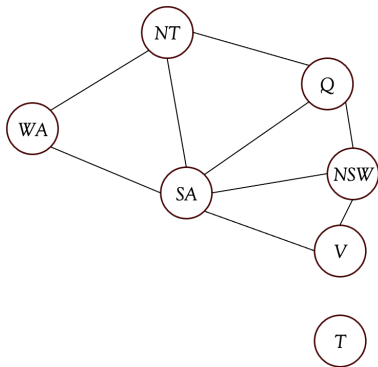
Tipos de restricciones

- Una **restricción unaria** restringe el valor de una sola variable.
Por ejemplo, para imponer que South Australia no se coloree de verde escribimos $\langle (SA), SA \neq green \rangle$
- Una **restricción binaria** relaciona dos variables.
Por ejemplo, $SA \neq NSW$
- Una restricción en la que participa un número arbitrario de variables se llama **restricción global**.
Por ejemplo, $AllDiff(F, T, U, W, R, O)$



Grafo de restricciones

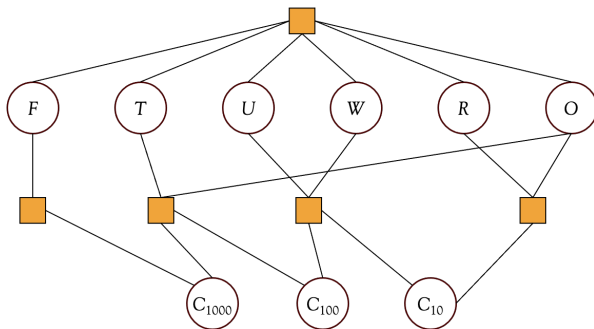
Cada variable se representa mediante un nodo, y cada restricción binaria se representa con un arco





Hipergrafo de restricciones

Los recuadros representan las restricciones globales





Propagación de Restricciones





Inferencia en CSPs

- Los problemas de búsqueda en el espacio de estados, solamente podemos buscar.
- CSPs tenemos dos posibilidades:
 1. Podemos realizar una búsqueda, escoger una asignación entre varias posibilidades.
 2. Realizar un tipo de inferencia, denominada propagación de restricciones, utilizando las restricciones para reducir el número de valores legales.



Inferencia en CSPs

- La propagación de restricciones se pueden intercalar con la búsqueda o como paso previo a la búsqueda.
- Dependiendo del problema con la propagación de restricciones es suficiente para encontrar una solución y no es necesario realizar la búsqueda.
- La idea es mantener la consistencia en el grafo de restricciones.



Consistencia de nodos

- Una variable es **nodo-consistente** si todos los valores del dominio de la variable satisfacen las **restricciones unarias** sobre dicha variable.

Por ejemplo, si tenemos la restricción unaria $\langle (SA), SA \neq \text{green} \rangle$, podemos hacer SA nodo-consistente quitando green de su dominio, lo que deja a SA con el dominio reducido red, blue

- Siempre es posible eliminar todas las restricciones unarias de un CSP ejecutando la consistencia de nodos.



Consistencia de arcos

- Una variable X_i es **arco-consistente** si, para cada valor del dominio de X_i y cada **restricción binaria** (X_i, X_j) , podemos encontrar al menos un valor en el dominio de X_j que satisface la restricción
- Una red es **arco-consistente** si toda variable es arco-consistente con las demás variables
- El siguiente algoritmo asegura la arco-consistencia de la variable X_i con respecto a otra variable X_j ; devuelve true si y sólo si se ha revisado el dominio de X_i .



Algoritmo de revisión de dominios



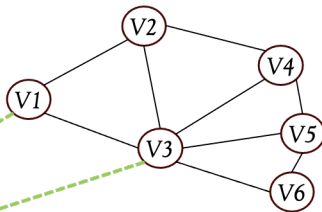
Algoritmo de revisión de dominios

El algoritmo revisa pares de
nodos, por ejemplo:

(V1,V3)

(V2,V3)

....



```
function REVISE(csp,  $X_i$ ,  $X_j$ ) returns true iff we revise the domain of  $X_i$   
  revised  $\leftarrow$  false  
  for each  $x$  in  $D_i$  do  
    if no value  $y$  in  $D_j$  allows  $(x,y)$  to satisfy the constraint between  $X_i$  and  $X_j$  then  
      delete  $x$  from  $D_i$   
      revised  $\leftarrow$  true  
  return revised
```



Algoritmo de revisión de dominios

Ejemplo:

$$X_i = V_1 X_j = V_2$$

$$V_1 = \{g, b\} V_2 = \{r\}$$

$$V_1 \neq V_2$$

Revised = false

$$D_i = \{g, b\} D_j = \{r\}$$

1.(g,r) Satisface las Restricciones

2.(b,r) Satisface las Restricciones

RETURN false

function REVISE(*csp*, X_i , X_j) **returns** true iff we revise the domain of X_i

revised $\leftarrow$ false

for each x **in** D_i **do**

if no value y in D_j allows (x,y) to satisfy the constraint between X_i and X_j **then**

 delete x from D_i

revised $\leftarrow$ true

return *revised*



Algoritmo de revisión de dominios

Ejemplo:

$$X_i = V_1 X_j = V_2$$

$$V_1 = \{g, b\} V_2 = \{b\}$$

$$V_1 \neq V_2$$

Revised = false

$$D_i = \{g, b\} D_j = \{b\}$$

1.(g,b) Satisface las Restricciones

2.(b,b) **NO** Satisface las Restricciones

a. Modifico D_i , $D_i = \{g, b\} D_i = \{g\}$

b. Revised = true

Salida:

$$V_1 = \{g\} V_2 = \{b\}$$

RETURN true

function REVISE(csp, X_i, X_j) **returns** true iff we revise the domain of X_i

revised $\leftarrow$ false

for each x **in** D_i **do**

if no value y in D_j allows (x, y) to satisfy the constraint between X_i and X_j **then**

delete x from D_i

revised $\leftarrow$ true

return *revised*



Algoritmo AC-3



Algoritmo AC-3

- El algoritmo más popular para asegurar la arco-consistencia en una red se llama AC-3
- Inicialmente el conjunto contiene todos los arcos del CSP
- A continuación extrae un arco cualquiera (X_i, X_j) del conjunto y hace X_i arco-consistente con respecto a X_j
Si esto reduce el dominio D_i , entonces añadimos al conjunto todos los arcos (X_k, X_i) , tales que X_k es vecino de X_i
- Si un dominio se reduce a nada, entonces el CSP original no tenía solución, y devolvemos false.
En otro caso obtenemos un CSP en el que es más fácil buscar.



Algoritmo AC-3

function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise

inputs: *csp*, a binary CSP with components (X , D , C)

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\text{queue})$

if REVISE(*csp*, X_i , X_j) **then**

if size of $D_i = 0$ **then return** false

for each X_k **in** $X_i.\text{NEIGHBORS} - \{X_j\}$ **do**

 add (X_k, X_i) to *queue*

return true

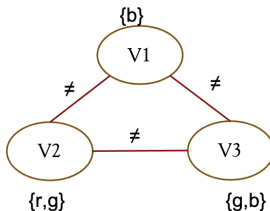


Ejemplo Algoritmo AC-3

Consideramos el siguiente problema:

- Tres variables: V_1, V_2, V_3
- Los dominios son: $D_1 = \{b\}, D_2 = \{r, g\}, D_3 = \{g, b\}$
- Las restricciones son: $V_1 \neq V_2, V_1 \neq V_3, V_2 \neq V_3$

Construimos el grafo de restricciones:





Ejemplo Algoritmo AC-3

function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise

inputs: *csp*, a binary CSP with components (X, D, C)

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\text{queue})$

if REVISE(*csp*, X_i , X_j) **then**

if size of $D_i = 0$ **then return** false

for each X_k **in** $X_i.\text{NEIGHBORS} - \{X_j\}$ **do**

 add (X_k, X_i) to *queue*

return true

function REVISE(*csp*, X_i , X_j) **returns** true iff we revise the domain of X_i

revised $\leftarrow$ false

for each x **in** D_i **do**

if no value y in D_j allows (x, y) to satisfy the constraint between X_i and X_j **then**

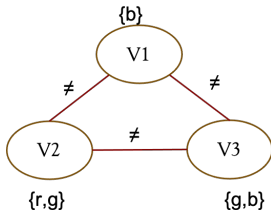
 delete x from D_i

revised $\leftarrow$ true

return *revised*



Ejemplo Algoritmo AC-3



Arcos	REVISE
(V1,V2)	
(V1,V3)	
(V2,V1)	
(V2,V3)	
(V3,V1)	
(V3,V2)	

function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise

inputs: *csp*, a binary CSP with components (X, D, C)

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

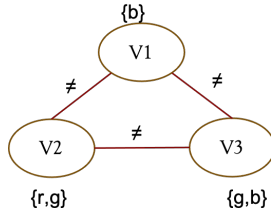
```
while queue is not empty do  
   $(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\text{queue})$   
  if REVISE(csp,  $X_i, X_j$ ) then  
    if size of  $D_i = 0$  then return false  
    for each  $X_k$  in  $X_i.\text{NEIGHBORS} - \{X_j\}$  do  
      add  $(X_k, X_i)$  to queue  
return true
```

1. Construimos una lista de arcos, todos los arcos del problema.

2. Vamos ejecutando el bucle While y llamamos al método REVISE



Ejemplo Algoritmo AC-3



Arcos	REVISE
(V1,V2)	{b} {r,g} FALSE
(V1,V3)	{b} {g,b} FALSE
(V2,V1)	
(V2,V3)	
(V3,V1)	
(V3,V2)	

function REVISE(*csp*, X_i , X_j) **returns** true iff we revise the domain of X_i
revised $\leftarrow$ false
for each x **in** D_i **do**
 if no value y in D_j allows (x,y) to satisfy the constraint between X_i and X_j **then**
 delete x from D_i
 revised $\leftarrow$ true
return *revised*

for aplica a cada elemento del primer dominio y debemos encontrar un elemento del segundo dominio que cumpla las restricciones

(b,r) cumple la restricción, FALSE

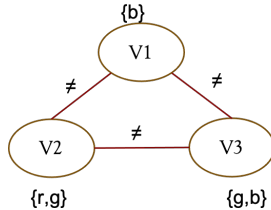
(b,g) cumple la restricción, FALSE

Al devolver FALSE no hacemos nada en AC-3

if REVISE(*csp*, X_i , X_j) **then**
 if size of D_i = 0 **then return** false
 for each X_k **in** X_i .NEIGHBORS - $\{X_j\}$ **do**
 add (X_k, X_i) to *queue*
return true



Ejemplo Algoritmo AC-3



```
function REVISE(csp,  $X_i$ ,  $X_j$ ) returns true iff we revise the domain of  $X_i$   
    revised  $\leftarrow$  false  
    for each  $x$  in  $D_i$  do  
        if no value  $y$  in  $D_j$  allows  $(x,y)$  to satisfy the constraint between  $X_i$  and  $X_j$  then  
            delete  $x$  from  $D_i$   
            revised  $\leftarrow$  true  
    return revised
```

for aplica a cada elemento del primer dominio y debemos encontrar un elemento del segundo dominio que cumpla las restricciones.

En los siguientes tenemos dos elementos

(r,b) cumple la restricción,
(g,b) cumple la restricción,

devuelve FALSE

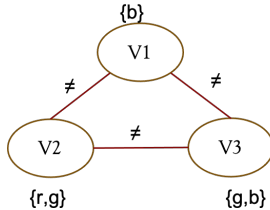
(r,g) cumple la restricción,
(g,b) cumple la restricción,

devuelve FALSE

Arcos	REVISE
(V1,V2)	{b} {r,g} FALSE
(V1,V3)	{b} {g,b} FALSE
(V2,V1)	{r,g} {b} FALSE
(V2,V3)	{r,g} {g,b} FALSE
(V3,V1)	
(V3,V2)	



Ejemplo Algoritmo AC-3



Arcos	REVISE
(V1,V2)	{b} {r,g} FALSE
(V1,V3)	{b} {g,b} FALSE
(V2,V1)	{r,g} {b} FALSE
(V2,V3)	{r,g} {g,b} FALSE
(V3,V1)	{g,b} {b} TRUE
(V3,V2)	

```

function REVISE(csp,  $X_i$ ,  $X_j$ ) returns true iff we revise the domain of  $X_i$ 
    revised  $\leftarrow$  false
    for each  $x$  in  $D_i$  do
        if no value  $y$  in  $D_j$  allows  $(x,y)$  to satisfy the constraint between  $X_i$  and  $X_j$  then
            delete  $x$  from  $D_i$ 
            revised  $\leftarrow$  true
    return revised
    
```

for aplica a cada elemento del primer dominio y debemos encontrar un elemento del segundo dominio que cumpla las restricciones.

(g,b) cumple la restricción,
(b,b) **NO** cumple la restricción,
Devuelve verdadero.

1. **Borramos b del dominio.** $D_3 \{g\}$
2. **Realizar tareas en AC-3**

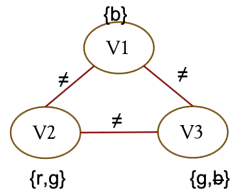
```

if REVISE(csp,  $X_i$ ,  $X_j$ ) then
    if size of  $D_i$  = 0 then return false
    for each  $X_k$  in  $X_i$ .NEIGHBORS -  $\{X_j\}$  do
        add  $(X_k, X_i)$  to queue
    return true
    
```



Ejemplo Algoritmo AC-3

Arcos	REVISE
(V1,V2)	{b} {r,g} FALSE
(V1,V3)	{b} {g,b} FALSE
(V2,V1)	{r,g} {b} FALSE
(V2,V3)	{r,g} {g,b} FALSE
(V3,V1)	{g,b} {b} TRUE
(V3,V2)	
(V1,V3)	
(V2,V3)	



Debido a que no está vacío el dominio, seguimos y añadimos nodos

```

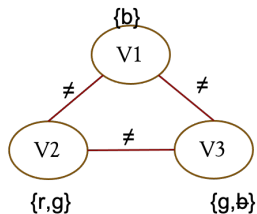
if REVISE( $csp, X_i, X_j$ ) then
  if size of  $D_i = 0$  then return false
  for each  $X_k$  in  $X_i.NEIGHBORS - \{X_j\}$  do
    add ( $X_k, X_i$ ) to queue
  return true

```



Ejemplo Algoritmo AC-3

Arcos	REVISE
(V1,V2)	{b} {r,g} FALSE
(V1,V3)	{b} {g,b} FALSE
(V2,V1)	{r,g} {b} FALSE
(V2,V3)	{r,g} {g,b} FALSE
(V3,V1)	{g,b} {b} TRUE
(V3,V2)	{g} {r,g} FALSE
(V1,V3)	
(V2,V3)	

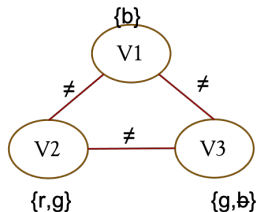


(g,r) cumple la restricción, FALSE



Ejemplo Algoritmo AC-3

Arcos	REVISE
(V1,V2)	{b} {r,g} FALSE
(V1,V3)	{b} {g,b} FALSE
(V2,V1)	{r,g} {b} FALSE
(V2,V3)	{r,g} {g,b} FALSE
(V3,V1)	{g,b} {b} TRUE
(V3,V2)	{g} {r,g} FALSE
(V1,V3)	
(V2,V3)	{r,g} {g} TRUE



(r,g) cumple la restricción,
 (g,g) **NO** cumple la restricción
 devuelve **TRUE**

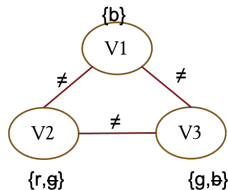
$D2 = \{r,g\} \rightarrow D2 = \{r\}$

Añadimos nuevos pares en AC-3



Ejemplo Algoritmo AC-3

Arcos	REVISE
(V1,V2)	{b} {r,g} FALSE
(V1,V3)	{b} {g,b} FALSE
(V2,V1)	{r,g} {b} FALSE
(V2,V3)	{r,g} {g,b} FALSE
(V3,V1)	{g,b} {b} TRUE
(V3,V2)	{g} {r,g} FALSE
(V1,V3)	
(V2,V3)	{r,g} {g} TRUE
(V1,V2)	{b} {r} FALSE
(V3,V2)	



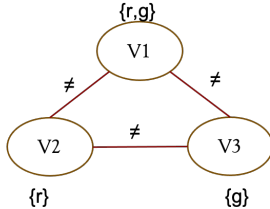
(b,r) cumple la restricción
devuelve FALSE

No tenemos más pares en la cola, con lo cual termina el Algoritmo AC-3, devolviendo los dominios reducidos:

$D1 = \{b\}$, $D2 = \{r\}$, $D3 = \{g\}$



Ejemplo 2 Algoritmo AC-3



Arcos	REVISE
(V1,V2)	{r,g} {r} TRUE V1={g}
(V1,V3)	{g} {g} TRUE V1={}
(V2,V1)	
(V2,V3)	
(V3,V1)	
(V3,V2)	
(V3,V1)	

function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise

inputs: *csp*, a binary CSP with components (X , D , C)

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

 (X_i , X_j) $\leftarrow$ REMOVE-FIRST(*queue*)

if REVISE(*csp*, X_i , X_j) **then**

if size of D_i = 0 **then return false**

for each X_k **in** X_i .NEIGHBORS - $\{X_j\}$ **do**

 add (X_k , X_i) to *queue*

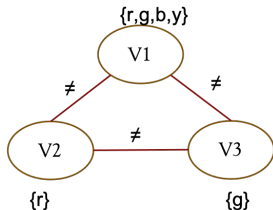
return true

$D1 = \{ \}$, $D2 = \{r\}$, $D3 = \{g\}$

No tiene solución debido
a que un dominio está vacío



Ejemplo 3 Algoritmo AC-3



$D1 = \{b,y\}, D2 = \{r\}, D3 = \{g\}$

El problema tiene dos soluciones:

$V1=b \ V2=r \ V3=g$

$V1=y \ V2=r \ V3=g$

Arcos	REVISE
(V1,V2)	$\{r,g,b,y\} \{r\}$ TRUE $V1=\{g,b,y\}$
(V1,V3)	$\{g,b,y\} \{g\}$ TRUE $V1=\{b,y\}$
(V2,V1)	$\{r\} \{b,y\}$ FALSE
(V2,V3)	$\{r\} \{g\}$ FALSE
(V3,V1)	$\{g\} \{b,y\}$ FALSE
(V3,V2)	$\{g\} \{r\}$ FALSE
(V3,V1)	$\{g\} \{b,y\}$ FALSE
(V2,V1)	$\{r\} \{b,y\}$ FALSE



Ejemplo Sudoku



Sudoku

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

(b)



Sudoku

- Un sudoku es un CSP con 81 variables, una para cada casilla.
- Las casillas vacías tienen un dominio $\{1,2,3,4,5,6,7,8,9\}$
Las casillas rellenas tienen un dominio con un único valor.
- Hay 27 *Alldiff* restricciones: una para cada fila, columna y casillas de 9 esquinas.
 - *Alldiff* (A1,A2,A3,A4,A5,A6,A7,A8,A9)
 - *Alldiff* (B1,B2,B3,B4,B5,B6,B7,B8,B9)
 - *Alldiff* (A1,A2,A3,B1,B2,B3,C1,C2,C3)
 - ...



Sudoku

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

Consideramos variable E6.

- Casillas: Podemos eliminar 1,7,2,8 del dominio.
 $D_{\text{casillas}} = \{3,4,5,6,9\}$
- Columna: Podemos eliminar 5,6,2,8,9,3 del dominio.
 $D_{\text{Columna}} = \{1,4,7\}$
- Fila: Podemos eliminar 7 y 8 del dominio
 $D_{\text{Fila}} = \{1,2,3,4,5,6,9\}$
- Domino E6 = $\{4\}$

Hemos asignado el valor a la variable E6 sin tener que aplicar el algoritmo de backtracking



Sudoku

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

Consideramos variable I9.

- Casillas: Podemos eliminar 3,5,9 del dominio.
 $DI9=\{1,2,4,6,7,8\}$
- Columna: Podemos eliminar 1,8,9 del dominio.
 $DI9=\{2,4,6,7\}$
- Fila: Podemos eliminar 1,5,3 del dominio
 $DI9=\{2,4,6,7\}$
- $DominioI9 = \{2,4,6,7\}$

En este caso no podemos asignar el valor a a la variable I9, con lo cual deberemos hacer uso del algoritmo de backtracking.



Algoritmos





Vuelta Atrás



Algoritmo vuelta atrás

- Muchos CSPs no se pueden resolver solamente por inferencia sobre las restricciones; llega un momento en el que hay que buscar una solución
- La búsqueda por vuelta atrás es una búsqueda primero en profundidad que en cada momento elige un valor para una sola variable, y vuelve atrás cuando una variable no tiene ningún valor legal que quede por probar.
- Debemos considerar las siguientes preguntas:
 1. ¿Qué variable debería ser asignada a continuación?
 2. ¿En qué orden deberíamos probar sus valores?
 3. ¿Qué inferencias deberían realizarse en cada paso?



Algoritmo vuelta atrás

function BACKTRACKING-SEARCH(*csp*) **returns** a solution, or failure
 return BACKTRACK({ }, *csp*)

function BACKTRACK(*assignment*, *csp*) **returns** a solution, or failure

if *assignment* is complete **then return** *assignment*

var $\leftarrow$ SELECT-UNASSIGNED-VARIABLE(*csp*)

1

for each *value* **in** ORDER-DOMAIN-VALUES(*var*, *assignment*, *csp*) **do**

2

if *value* is consistent with *assignment* **then**

 add {*var* = *value*} to *assignment*

inferences $\leftarrow$ INFERENCE(*csp*, *var*, *value*)

3

if *inferences* $\neq$ failure **then**

 add *inferences* to *assignment*

result $\leftarrow$ BACKTRACK(*assignment*, *csp*)

if *result* $\neq$ failure **then**

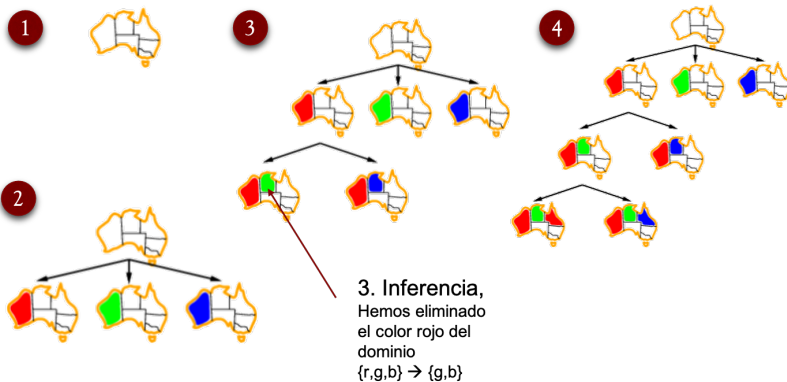
return *result*

 remove {*var* = *value*} and *inferences* from *assignment*

return failure



Ejemplo Algoritmo vuelta atrás





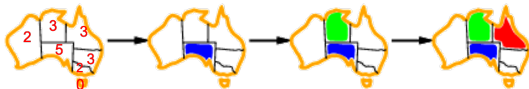
1. Ordenación de las variables

- Habitualmente se elige la variable que tenga el menor número de valores legales. Esto es lo que se llama el **heurístico del mínimo número de valores restantes**.

(**minimum remaining values heuristic, MRV**)

$WA = \{\text{red, green, blue}\}, Q = \{\text{red}\}$

- También es posible emplear el **heurístico del grado (degree heuristic, DEG)**, que elige la variable que interviene en el mayor número de restricciones con otras variables no asignadas. Esta manera de elegir las variables intenta obtener un fallo tan pronto como sea posible para podar secciones más grandes del árbol de búsqueda rápidamente

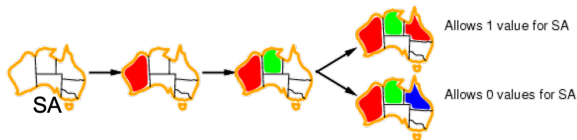




2.Ordenación de los valores

En algunos casos el **heurístico del valor menos restrictivo (least constraining value, LCV)** puede resultar útil

- Elige un valor que restrinja menos las opciones de los demás variables vecinas en el grafo de restricciones.
- Este heurístico intenta obtener una solución tan pronto como sea posible eligiendo primero los valores más verosímiles.





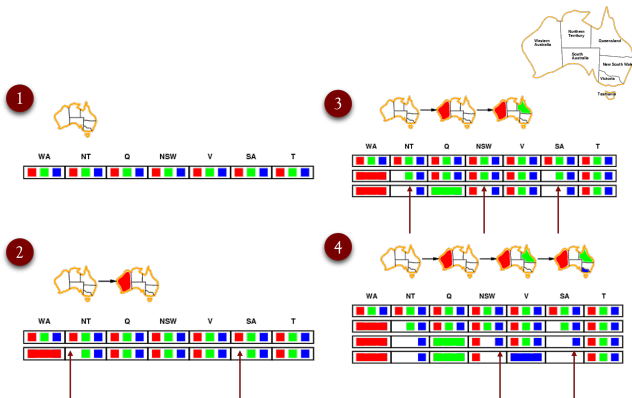
3. Intercalando búsqueda e inferencias

- Cada vez que hacemos una elección de un valor para una variable, intentamos inferir nuevas reducciones de dominio en las variables vecinas
- Una de las estrategias más sencillas es la comprobación hacia delante (**forward checking**): Cada vez que se asigna una variable X , para cada variable no asignada Y que está conectada a X mediante una restricción, borramos del dominio de Y los valores que son inconsistentes con el valor elegido para X
- Para muchos problemas la búsqueda es más efectiva si combinamos la heurística MRV con la comprobación hacia delante.



3. Intercalando búsqueda e inferencias

Forward Checking





3. Intercalando búsqueda e inferencias

Forward Checking y MRV (heurístico del mínimo número de valores restantes)

1



WA	NT	Q	NSW	V	SA	T

2



WA	NT	Q	NSW	V	SA	T





3. Intercalando búsqueda e inferencias

Forward checking N Reinas

1

	Q _A	Q _B	Q _C	Q _D
Row 1				
Row 2				
Row 3				
Row 4				

1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

Posible valores

2

	Q _A	Q _B	Q _C	Q _D
Row 1	Q			
Row 2				
Row 3				
Row 4				

1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

Posible valores



3. Intercalando búsqueda e inferencias

Forward checking N Reinas

3

	Q _A	Q _B	Q _C	Q _D
Row 1	Q			
Row 2				
Row 3				
Row 4				

1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

Posible valores

4

	Q _A	Q _B	Q _C	Q _D
Row 1	Q			
Row 2				
Row 3		Q		
Row 4				

1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

Posible valores



3. Intercalando búsqueda e inferencias

Forward checking N Reinas

5

	Q _A	Q _B	Q _C	Q _D
Row 1	Q			
Row 2				
Row 3		Q		
Row 4				

1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4

Posible valores

Sin Solución

6

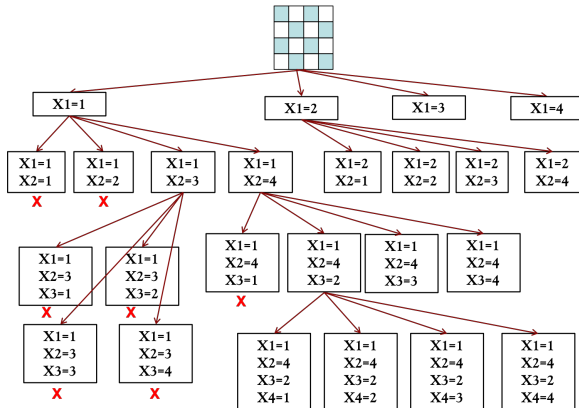
	Q _A	Q _B	Q _C	Q _D
Row 1	Q			
Row 2				
Row 3				
Row 4		Q		

1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4

Posible valores

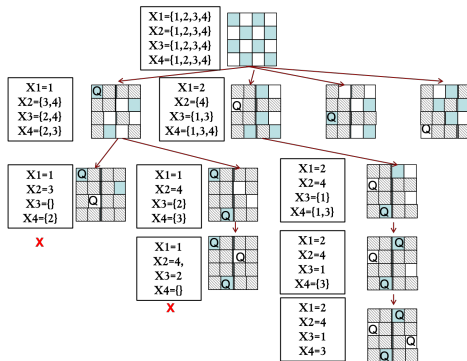


Backtracking N Reinas





Backtracking y Forward checking N Reinas





Búsqueda Local



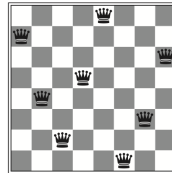
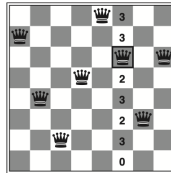
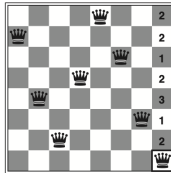
Búsqueda Local

- Los algoritmos de búsqueda local pueden ser efectivos para resolver algunos problemas de CSP.
- El estado inicial establece un valor a cada variable.
- Y la búsqueda cambia el valor de una variable cada paso.
- La solución inicial normalmente no cumplirá las restricciones, el objetivo es que se cumplan las restricciones.



Búsqueda Local

- Para escoger variable, la mejor heurística es seleccionar aquella que tiene un menor número de conflictos con otras variables (Min-conflicts heurístico)





Búsqueda Local

function MIN-CONFLICTS(csp, max_steps) **returns** a solution or failure

inputs: csp , a constraint satisfaction problem

max_steps , the number of steps allowed before giving up

$current \leftarrow$ an initial complete assignment for csp

for $i = 1$ to max_steps **do**

if $current$ is a solution for csp **then return** $current$

$var \leftarrow$ a randomly chosen conflicted variable from $csp.VARIABLES$

$value \leftarrow$ the value v for var that minimizes CONFLICTS($var, v, current, csp$)

 set $var = value$ in $current$

return *failure*



Resumen





Resumen

Objetivo obtener las asignaciones de los valores de los dominios a las variables que cumplan las restricciones.

1. Identificar
Variables V_1, V_2, V_3
Dominios D_1, D_2, D_3
Restricciones p.ej. $V_1 \neq V_2$
2. Intentar reducir los dominios. Aplicar las restricciones a los dominios originales.
 - 2.1 Eliminación valores dominios.
 - 2.2 Aplicación algoritmo AC-3.
3. Los dominios se han quedado vacío. **No tengo solución**
4. Si los dominios son pequeños y dominios no vacíos. **Solución problema**
5. Los dominios aún reduciendo son muy grandes pasamos a Backtracking para buscar solución al problema.
 - 5.1 Establecer un orden de las variables.
 - 5.2 Establecer un orden de los dominios.
 - 5.3 Realizar inferencias, reducir los dominios una vez que tenga asignación



That's all folks!

